

杂项

Morning Flower and Evening Oath

XCPC vector 模板分册

目录

使用约定	ii
第一章 Dancing Links	1
1.1 精确覆盖 (Dancing Links)	1
1.2 最少行重复覆盖 (Dancing Links)	5
第二章 离线算法	10
2.1 CDQ 分治	10
2.1.1 静态三维偏序计数	10
2.1.2 三维最长链、计数与方案	12
2.2 莫队算法	17
2.2.1 带修改莫队	17
2.2.1.1 带修改的莫队	17
2.2.2 莫队二次离线	20
2.2.2.1 莫队二次离线：异或位数配对	20
第三章 配套知识与验证范围	23
3.1 带修莫队：时间轴与区间状态	23
3.2 莫队二次离线：前缀事件与自身扣除	24
第一部分 跨类依赖	25
附录 A 数论	26
A.1 模算术简介	26
A.1.1 静态模整数	26
A.2 卢卡斯定理	28
A.2.1 Lucas：小素数模的大下标组合数	28
附录 B 组合数学	30
B.1 排列组合	30
B.1.1 阶乘组合数	30
附录 C 树状数组	32
C.1 树状数组第 k 小与组合依赖	32

使用约定

C++20，默认包含 `bits/stdc++.h` 与 `cassert`，并使用 `std` 命名空间。下标、区间及数值范围以各条目说明为准。DFS 保持递归，需满足题目栈空间条件。

本册主目录沿用 OI Wiki 官方导航的大、中、小层级，各页下列本库实现。PBDS 与 GNU rope 按队内约定纳入平衡树。组合条目与相关挂靠仍见仓库分类审计。跨类引用的已收录组件附在依赖部分；源头文件中的依赖仍须一并检查。

本册尚非全量完成稿；传统版评测仅为历史档案，当前只维护 `vector` 代码。

第一章 Dancing Links

1.1 精确覆盖 (Dancing Links)

ExactCover(m,rows): 列 0..m-1, 各稀疏行内列号互异、无需排序; 行可重复或为空, 保留原 0-based 行号。solve() 返回 optional<vector<int>>, 无解为 nullopt, m=0 为有值的空解。只求任意首解, 不保证最少行数。成功/失败均恢复链与列计数, 可重复调用及复制; 实现字段不可外改, 移后源对象仅销毁或重新赋值。m 和行数 <INT_MAX, 1+m+ 非零数 ≤INT_MAX。构造 O(行数 +m+ 非零数), 空间 O(m+ 非零数), 递归深度 ≤m, 搜索最坏指数。

```
1 struct ExactCover
2 {
3     struct Node { int l, r, u, d, c, row; };
4     int m;
5     vector<Node> a;
6     vector<int> s, ans;
7
8     ExactCover(int columns, const vector<vector<int>> &rows) : m(columns)
9     {
10         assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
11         size_t z = size_t(m) + 1;
12         for (const auto &r : rows)
13         {
14             assert(r.size() <= size_t(INT_MAX) - z);
15             z += r.size();
16         }
17         a.reserve(z);
18         s.assign(m + 1, 0);
19         for (int c = 0; c <= m; c++)
20             a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});
```

稀疏行建链与删除（接上页同一结构体）：

```

21     vector<int> seen(m, -1);
22     for (int i = 0; i < int(rows.size()); i++)
23     {
24         int h = -1;
25         for (int col : rows[i])
26         {
27             assert(0 <= col && col < m && seen[col] != i);
28             seen[col] = i;
29             int c = col + 1, x = a.size(), u = a[c].u;
30             a.push_back({x, x, u, c, c, i});
31             a[u].d = a[c].u = x;
32             s[c]++;
33             if (h == -1) h = x;
34             else
35             {
36                 int l = a[h].l;
37                 a[x].l = l; a[x].r = h;
38                 a[l].r = a[h].l = x;
39             }
40         }
41     }
42 }
43
44 void cover(int c)
45 {
46     a[a[c].l].r = a[c].r;
47     a[a[c].r].l = a[c].l;
48     for (int i = a[c].d; i != c; i = a[i].d)
49         for (int j = a[i].r; j != i; j = a[j].r)
50         {
51             a[a[j].u].d = a[j].d;
52             a[a[j].d].u = a[j].u;
53             s[a[j].c]--;
54         }
55 }

```

逆序恢复与搜索（接上页同一结构体）：

```

57 void uncover(int c)
58 {
59     for (int i = a[c].u; i != c; i = a[i].u)
60         for (int j = a[i].l; j != i; j = a[j].l)
61             {
62                 s[a[j].c]++;
63                 a[a[j].u].d = a[a[j].d].u = j;
64             }
65     a[a[c].l].r = a[a[c].r].l = c;
66 }
67
68 bool dfs()
69 {
70     int c = a[0].r;
71     if (!c) return true;
72     for (int j = a[c].r; j; j = a[j].r)
73         if (s[j] < s[c]) c = j;
74     cover(c);
75     for (int i = a[c].d; i != c; i = a[i].d)
76     {
77         ans.push_back(a[i].row);
78         for (int j = a[i].r; j != i; j = a[j].r) cover(a[j].c);
79         bool ok = dfs();
80         for (int j = a[i].l; j != i; j = a[j].l) uncover(a[j].c);
81         if (ok)
82             {
83                 uncover(c);
84                 return true;
85             }
86         ans.pop_back();
87     }
88     uncover(c);
89     return false;
90 }
91
92 optional<vector<int>>> solve()
93 {
94     ans.clear();
95     ans.reserve(m);
96     if (!dfs()) return nullopt;
97     return ans;
98 }
99 };

```

模板题使用：Luogu P4929 / 精确覆盖

题意：读入 $N \times M$ 的 01 矩阵， $N, M \leq 500$ 且非零数 ≤ 5000 ；每列恰被选择行覆盖一次。稀疏建模后输出原 1-based 行号，任一可行解均可；无解输出 No Solution!。

所需模板：ExactCover（第 1 页，1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P4929>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> rows(n);
8     for (int i = 0; i < n; i++)
9         for (int j = 0, x; j < m; j++)
10            {
11                cin >> x;
12                if (x) rows[i].push_back(j);
13            }
14     ExactCover dlx(m, rows);
15     auto ans = dlx.solve();
16     if (!ans) cout << "No Solution!\n";
17     else
18     {
19         for (int i : *ans) cout << i + 1 << ' ';
20         cout << '\n';
21     }
22 }
```

1.2 最少行重复覆盖 (Dancing Links)

MinimumCover(m, rows), 列号 0..m-1, 每行内部互异, 允许空行/重复行。solve() 返回最少行数的一组原 0-based 行号; nullopt 无解, 有值空向量表示 m=0。每列至少一次, 允许重叠; 不含权值、覆盖次数或全解枚举。 $0 \leq m < \text{INT_MAX}$, 行数 $< \text{INT_MAX}$, $m+1 + \text{非零数} \leq \text{INT_MAX}$; 前提由调用方保证。构造 $O(\text{行数} + m + \text{非零数})$, 空间 $O(m + \text{非零数})$; 递归搜索最坏指数、深度 $\leq m$, 每状态 $O(m + \text{非零数})$ 。成功失败均恢复链, 可重复求解或复制; 重建用重新赋值。字段及辅助方法不作为可修改接口, 移动后源对象只销毁或赋值。下界选取不能被同一行共同覆盖的列, 至少还需同样多行; 按最少候选列搜索, 逆序恢复只改左右链。

```

1 struct MinimumCover
2 {
3     struct Node
4     {
5         int l, r, u, d, c, row;
6     };
7     int m, best = 0;
8     vector<Node> a;
9     vector<int> s, path, ans;
10    vector<unsigned char> mark;
11
12    MinimumCover(int columns, const vector<vector<int>> &rows) : m(columns)
13    {
14        assert(0 <= m && m < INT_MAX && rows.size() < INT_MAX);
15        size_t z = size_t(m) + 1;
16        for (const auto &r : rows)
17        {
18            assert(r.size() <= size_t(INT_MAX) - z);
19            z += r.size();
20        }
21        a.reserve(z);
22        s.assign(m + 1, 0);
23        mark.resize(m + 1);
24        path.reserve(m);
25        ans.reserve(m);
26        for (int c = 0; c <= m; c++)
27            a.push_back({c ? c - 1 : m, c == m ? 0 : c + 1, c, c, c, -1});

```

稀疏建链与删列（接上页同一结构体）：

```

28     vector<int> seen(m, -1);
29     for (int i = 0; i < int(rows.size()); i++)
30     {
31         int h = -1;
32         for (int col : rows[i])
33         {
34             assert(0 <= col && col < m && seen[col] != i);
35             seen[col] = i;
36             int c = col + 1, x = a.size(), u = a[c].u;
37             a.push_back({x, x, u, c, c, i});
38             a[u].d = x;
39             a[c].u = x;
40             s[c]++;
41             if (h == -1) h = x;
42             else
43             {
44                 int l = a[h].l;
45                 a[x].l = l;
46                 a[x].r = h;
47                 a[l].r = x;
48                 a[h].l = x;
49             }
50         }
51     }
52 }
53
54 // x is a selected row node, not a column header.
55 void remove(int x)
56 {
57     for (int i = a[x].d; i != x; i = a[i].d)
58     {
59         a[a[i].l].r = a[i].r;
60         a[a[i].r].l = a[i].l;
61     }
62 }

```

逆序恢复与可采纳下界（接上页同一结构体）：

```
64 void restore(int x)
65 {
66     for (int i = a[x].u; i != x; i = a[i].u)
67     {
68         a[a[i].l].r = i;
69         a[a[i].r].l = i;
70     }
71 }
72
73 int lower_bound()
74 {
75     fill(mark.begin(), mark.end(), 0);
76     for (int c = a[0].r; c; c = a[c].r) mark[c] = 1;
77     int bound = 0;
78     for (int c = a[0].r; c; c = a[c].r)
79     {
80         if (!mark[c]) continue;
81         bound++;
82         mark[c] = 0;
83         for (int i = a[c].d; i != c; i = a[i].d)
84             for (int j = a[i].r; j != i; j = a[j].r)
85                 mark[a[j].c] = 0;
86     }
87     return bound;
88 }
```

最优搜索与求解入口（接上页同一结构体）：

```
90     void dfs()
91     {
92         int depth = path.size(), c = a[0].r;
93         if (!c)
94         {
95             if (depth < best)
96             {
97                 best = depth;
98                 ans = path;
99             }
100            return;
101        }
102        if (lower_bound() >= best - depth) return;
103        for (int j = a[c].r; j; j = a[j].r)
104            if (s[j] < s[c]) c = j;
105        for (int i = a[c].d; i != c; i = a[i].d)
106        {
107            path.push_back(a[i].row);
108            remove(i);
109            for (int j = a[i].r; j != i; j = a[j].r) remove(j);
110            dfs();
111            for (int j = a[i].l; j != i; j = a[j].l) restore(j);
112            restore(i);
113            path.pop_back();
114        }
115    }
116
117    optional<vector<int>> solve()
118    {
119        path.clear();
120        ans.clear();
121        best = m + 1;
122        dfs();
123        if (best > m) return nullopt;
124        return ans;
125    }
126};
```

接口用法演示：最少行重复覆盖（本地接口示例）

演示约定：读入 n 个稀疏集合和 m 列；每行先给元素数，再给互异的 0-based 列号。输出最少选行数及原 0-based 行号，使每列至少出现一次；无解输出 -1， $m=0$ 输出 0 与空方案。此为接口演示，不冒充线上模板题。

所需模板：MinimumCover（第 5 页，1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://oi-wiki.org/search/dlx/>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, m;
6     cin >> n >> m;
7     vector<vector<int>> rows(n);
8     for (auto &row : rows)
9     {
10         int k;
11         cin >> k;
12         row.resize(k);
13         for (int &c : row) cin >> c;
14     }
15     MinimumCover cover(m, rows);
16     auto ans = cover.solve();
17     if (!ans) cout << -1 << '\n';
18     else
19     {
20         cout << ans->size() << '\n';
21         for (int row : *ans) cout << row << ' ';
22         cout << '\n';
23     }
24 }
```

第二章 离线算法

2.1 CDQ 分治

2.1.1 静态三维偏序计数

代码依赖：Fenwick (第 32 页, C.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

`count(points)` 返回每个原输入位置被其他点逐坐标 $\leq$ 的次数，重复坐标仍是不同点，排除自身。输入 `array<long long,3>`，完整 signed64 坐标，空输入返回空；输入不变。内部按三维字典序合并重合点，压缩第三维，CDQ 归并与 Fenwick 回滚。 $O(n \log^2(n+1))$ 时间、 $O(n)$ 空间、 $O(\log(n+1))$ 递归； n 须 $\leq \text{INT_MAX}/2$ 且内存可分配，保证共享 Fenwick 的 `int` 步进不溢出。仅静态无权重偏序，不含严格不等号、动态更新或 CDQ 递推。

```
1 struct Dominance3D
2 {
3     static vector<int> count(const vector<array<long long, 3>> &p)
4     {
5         assert(p.size() <= INT_MAX / 2);
6         int n = int(p.size());
7         vector<int> order(n), group(n), w;
8         iota(order.begin(), order.end(), 0);
9         sort(order.begin(), order.end(), [&](int i, int j) { return p[i] < p[j]; });
10        vector<array<long long, 3>> v;
11        for (int i : order)
12        {
13            if (v.empty() || v.back() != p[i])
14            {
15                v.push_back(p[i]);
16                w.push_back(0);
17            }
18            ++w.back();
19            group[i] = int(v.size()) - 1;
20        }
21        int m = int(v.size());
22        vector<long long> z;
23        for (auto q : v) z.push_back(q[2]);
24        sort(z.begin(), z.end());
25        z.erase(unique(z.begin(), z.end()), z.end());
26        struct Node
27        {
28            long long y;
29            int z, w, id;
30        };
31        vector<Node> a(m), tmp(m);
32        vector<int> cnt(m), ans(n);
33        for (int i = 0; i < m; ++i)
34            a[i] = {v[i][1], int(lower_bound(z.begin(), z.end(), v[i][2]) -
35                                   z.begin()) + 1, w[i], i};
36        Fenwick<int> bit(int(z.size()));
```

三维偏序的 CDQ 归并与原序返回（接上页同一 count 函数）：

```

37     auto cdq = [&](auto &&self, int l, int r) -> void
38     {
39         if (r - l <= 1) return;
40         int mid = l + (r - l) / 2;
41         self(self, l, mid);
42         self(self, mid, r);
43         int i = l, k = 1;
44         for (int j = mid; j < r; ++j)
45         {
46             while (i < mid && a[i].y <= a[j].y)
47             {
48                 bit.add(a[i].z, a[i].w);
49                 tmp[k++] = a[i++];
50             }
51             cnt[a[j].id] += bit.sum(a[j].z);
52             tmp[k++] = a[j];
53         }
54         for (int t = l; t < i; ++t) bit.add(a[t].z, -a[t].w);
55         while (i < mid) tmp[k++] = a[i++];
56         copy(tmp.begin() + 1, tmp.begin() + r, a.begin() + 1);
57     };
58     cdq(cdq, 0, m);
59     for (int i = 0; i < n; ++i)
60         ans[i] = cnt[group[i]] + w[group[i]] - 1;
61     return ans;
62 }
63 };

```

模板题使用：Luogu P3810 【模板】三维偏序 / 陌上花开

题意： $1 \leq n \leq 100000$ ， $1 \leq$ 三个坐标 $\leq k \leq 200000$ 。对每个输入点统计其他点三个坐标均 $\leq$ 自身的次数，输出次数为 $0..n-1$ 的点数直方图。重复位置分别计数且各自排除自己。 k 只约束题面坐标，库内部压缩，无需外部初始化；空输入和 signed64 极值另属 API 扩展。

所需模板：Fenwick（第 32 页，C.1 节）、Dominance3D（第 10 页，2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P3810>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, k;
6     cin >> n >> k;
7     vector<array<long long, 3>> p(n);
8     for (auto &[x, y, z] : p) cin >> x >> y >> z;
9     vector<int> hist(n);
10    for (int x : Dominance3D::count(p)) ++hist[x];
11    for (int x : hist) cout << x << '\n';
12 }

```

2.1.2 三维最长链、计数与方案

Chain3D(points, mod, strict=true), 坐标 array<long long,3>, 输入不变。

mod 为 1..INT_MAX, $n \leq \text{INT_MAX}/2$ 且内存可分配。

strict=true 要求每一步三个坐标都严格增加; false 为三坐标不减, 并固定按 (x,y,z, 原输入编号) 递增的规范顺序, 重合点可依编号串联, 不把相同坐标的排列重复计数。length/count 为最长链长度/模方案数; 空输入为 0 和 1%mod。len/ways/parent 按 0-based 原输入编号存每点结尾的最优状态与前驱, 无前驱为-1; path() 返回一条最长链原编号。并列时逐步选编号较小的前驱/终点, 不保证整条路径字典序最小; 计数为 0 模 mod 不表示无方案。CDQ 左递归 → 转移 → 右递归, 严格模式整组相同 x 不互转; y 扫描、z 压缩树状数组维护最大长度和同长计数。 $O(n \log^2(n+1))$ 时间、 $O(n)$ 空间、 $O(\log(n+1))$ 递归; 坐标只比较, 支持完整 signed64。所有成员只读, 重新赋值可重算。

```

1 struct Chain3D
2 {
3     using Point = array<long long, 3>;
4
5     struct State
6     {
7         int len = 0, ways = 0, end = -1;
8     };
9
10    vector<int> len, ways, parent;
11    int length = 0, count = 0, last = -1;
12
13    Chain3D(const vector<Point> &p, int mod, bool strict = true)
14    {
15        assert(mod > 0 && p.size() <= INT_MAX / 2);
16        int n = p.size();
17        count = 1 % mod;
18        len.assign(n, 1);
19        ways.assign(n, 1 % mod);
20        parent.assign(n, -1);
21        if (!n) return;

```

坐标顺序、分组与状态合并（接上页同一结构体）：

```

22     vector<int> order(n), z(n), cut{0};
23     vector<long long> values;
24     iota(order.begin(), order.end(), 0);
25     sort(order.begin(), order.end(), [&](int x, int y)
26     {
27         if (p[x] != p[y]) return p[x] < p[y];
28         return x < y;
29     });
30     for (int i = 1; i < n; i++)
31     {
32         if (!strict || p[order[i]][0] != p[order[i - 1]][0]) cut.push_back(i);
33     }
34     cut.push_back(n);
35     for (auto q : p) values.push_back(q[2]);
36     sort(values.begin(), values.end());
37     values.erase(unique(values.begin(), values.end()), values.end());
38     for (int i = 0; i < n; i++)
39     {
40         z[i] = lower_bound(values.begin(), values.end(), p[i][2]) - values.begin() + 1;
41     }
42     vector<State> bit(values.size() + 1);
43     auto join = [&](State x, State y)
44     {
45         if (x.len != y.len) return x.len > y.len ? x : y;
46         x.ways = ((long long)x.ways + y.ways) % mod;
47         x.end = min(x.end, y.end);
48         return x;
49     };

```

先转移再右递归（同一构造函数内）（接上页同一结构体）：

```

50     auto cdq = [&](auto &&self, int l, int r) -> void
51     {
52         if (r - l <= 1) return;
53         int mid = l + (r - l) / 2;
54         self(self, l, mid);
55         {
56             vector<int> a(order.begin() + cut[l], order.begin() + cut[mid]);
57             vector<int> b(order.begin() + cut[mid], order.begin() + cut[r]);
58             auto by_y = [&](int x, int y)
59             {
60                 return p[x][1] < p[y][1];
61             };
62             sort(a.begin(), a.end(), by_y);
63             sort(b.begin(), b.end(), by_y);
64             int i = 0;
65             for (int v : b)
66             {
67                 while (i < (int)a.size())
68                 {
69                     int u = a[i];
70                     if (strict ? p[u][1] >= p[v][1] : p[u][1] > p[v][1]) break;
71                     i++;
72                     for (int t = z[u]; t < (int)bit.size(); t += t & -t)
73                     {
74                         bit[t] = join(bit[t], {len[u], ways[u], u});
75                     }
76                 }
77                 State best;
78                 for (int t = z[v] - strict; t > 0; t -= t & -t)
79                 {
80                     best = join(best, bit[t]);
81                 }
82                 if (!best.len) continue;
83                 if (best.len + 1 > len[v])
84                 {
85                     len[v] = best.len + 1;
86                     ways[v] = best.ways;
87                     parent[v] = best.end;
88                 }
89                 else if (best.len + 1 == len[v])
90                 {
91                     ways[v] = ((long long)ways[v] + best.ways) % mod;
92                     parent[v] = min(parent[v], best.end);
93                 }
94             }
95             for (int j = 0; j < i; j++)
96             {
97                 for (int t = z[a[j]]; t < (int)bit.size(); t += t & -t) bit[t] = State{};
98             }
99         }
100         self(self, mid, r);
101     };

```

汇总答案与路径恢复（接上页同一结构体）：

```

102     cdq(cdq, 0, cut.size() - 1);
103     State best;
104     for (int i = 0; i < n; i++) best = join(best, {len[i], ways[i], i});
105     length = best.len;
106     count = best.ways;
107     last = best.end;
108 }
109
110 vector<int> path() const
111 {
112     vector<int> ans;
113     for (int u = last; u != -1; u = parent[u]) ans.push_back(u);
114     reverse(ans.begin(), ans.end());
115     return ans;
116 }
117 };

```

接口用法演示：三维最长链与原编号方案（接口演示）

演示约定：输入 $n, \text{mod}, \text{strict}$ 及 n 个三维整数点。 $\text{strict}=1$ 为三维都严格增加； $\text{strict}=0$ 为三维不减，重合点按原编号串联，不重复计算它们的排列。输出全局长度/模方案数、一条原编号路径，以及各原输入点的结尾长度/模方案数/前驱编号。空输入全局为 0 和 $1\% \text{mod}$ ，路径为空； $\text{mod}=1$ 时计数 0 也能恢复路径。

所需模板：Chain3D（第 12 页，2.1.2 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源：<https://oi-wiki.org/misc/cdq-divide/>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n, mod, strict;
6     cin >> n >> mod >> strict;
7     vector<Chain3D::Point> p(n);
8     for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
9     Chain3D chain(p, mod, strict);
10    cout << chain.length << ' ' << chain.count << '\n';
11    for (int id : chain.path()) cout << id << ' ';
12    cout << '\n';
13    for (int i = 0; i < n; i++)
14    {
15        cout << chain.len[i] << ' ' << chain.ways[i] << ' ' << chain.parent[i] << '\n';
16    }
17 }

```

接口用法演示：三维弱链计数 (kuangbin HDU4742 源码模型)

演示约定：输入 T 组，每组给 $n \geq 1$ 和 n 个三维整数点，输出按三维弱偏序及重合点固定顺序得到的最长链长度、方案数模 2^{30} 。这是原稿源码的闭边界模型；其文字描述却写三个严格小于，原 HDU4742 题面/约束/协议尚未核验，不视为正式模板题或已完成 OJ 适配。

所需模板：Chain3D (第 12 页, 2.1.2 节)。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

来源： <https://github.com/kuangbin/ACM-ICPC>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int tests;
6     cin >> tests;
7     while (tests--)
8     {
9         int n;
10        cin >> n;
11        vector<Chain3D::Point> p(n);
12        for (auto &v : p) cin >> v[0] >> v[1] >> v[2];
13        Chain3D chain(p, 1 << 30, false);
14        cout << chain.length << ' ' << chain.count << '\n';
15    }
16 }
```

2.2 莫队算法

2.2.1 带修改莫队

2.2.1.1 带修改的莫队

ModifiedMo(n, m) 保存离线询问, $n > 0$, m 为修改总数。add(l, r, t) 登记第 t 次修改完成后的 $[l, r]$, 返回从 0 开始的询问编号; $0 \leq l \leq r \leq n$, $0 \leq t \leq m$, 询问数小于 INT_MAX。run(add, del, chg, ans, block=0) 排序并移动三个指针。add(p)/del(p) 按下标更新当前区间统计; chg($i, forward, l, r$) 将编号 i 的修改前进或回退, 并同步修正当前 $[l, r]$ 内的贡献; ans(id) 记录结果。修改编号从 0 开始, 前进第 i 次修改时原时间为 i , 回退时原时间为 $i+1$ 。回调初始状态必须为原数组、空区间、时间 0; 重复 run 前由调用方重置状态。状态不会自动恢复, 原询问顺序不变。仅适用于可增删维护的区间统计, 需区分左右端操作时另行拆分回调。默认块长约 $n^{2/3}$, 也可显式指定正块长。 n 、询问数、修改数同阶且回调 $O(1)$ 时总时间 $O(n^{5/3})$, 调度器空间 $O(q)$ 。

```

1 struct ModifiedMo
2 {
3     struct Query
4     {
5         int l, r, t, id;
6     };
7
8     int n, m;
9     vector<Query> q;
10
11     ModifiedMo(int n, int m) : n(n), m(m) { assert(n > 0 && m >= 0); }
12
13     int add(int l, int r, int t)
14     {
15         assert(0 <= l && l <= r && r <= n);
16         assert(0 <= t && t <= m);
17         int id = q.size();
18         q.push_back({l, r, t, id});
19         return id;
20     }

```

```

22  template <class Add, class Del, class Chg, class Ans>
23  void run(Add add, Del del, Chg chg, Ans ans, int block = 0) const
24  {
25      if (q.empty()) return;
26      if (block == 0) block = max(1, int(pow(n, 2.0 / 3)));
27      assert(block > 0);
28      auto ord = q;
29      auto cmp = [&](const Query &a, const Query &b)
30      {
31          int x = a.l / block, y = b.l / block;
32          if (x != y) return x < y;
33          int p = a.r / block, q = b.r / block;
34          if (p != q) return (x & 1) ? p > q : p < q;
35          return ((x ^ p) & 1) ? a.t > b.t : a.t < b.t;
36      };
37      sort(ord.begin(), ord.end(), cmp);
38      int l = 0, r = 0, t = 0;
39      for (auto q : ord)
40      {
41          while (t < q.t)
42          {
43              chg(t, true, l, r);
44              t++;
45          }
46          while (t > q.t)
47          {
48              t--;
49              chg(t, false, l, r);
50          }
51          while (l > q.l) add(--l);
52          while (r < q.r) add(r++);
53          while (l < q.l) del(l++);
54          while (r > q.r) del(--r);
55          ans(q.id);
56      }
57  }
58  };

```

模板题使用：Luogu P1903

题意：单点改色、区间不同颜色数， $n, m \leq 133333$ ，颜色 $1..1000000$ 。先读完操作，`cur` 仅用于记录每次修改前后的颜色，`a` 保持初始数组。题目闭区间 $[L, R]$ 转为 $[L-1, R)$ ，询问时间是此前修改数。`chg` 必须先删旧贡献、改 `a[p]`、再加新贡献，回退使用 `pre`。按登记顺序输出。三维莫队，回调 $O(1)$ ，同阶规模时 $O(n^{5/3})$ 。

所需模板：ModifiedMo（第 17 页，2.2.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P1903>

Luogu P1903: 使用代码

```

1 int main()
2 {
3     int n, m;
4     scanf("%d%d", &n, &m);
5     vector<int> a(n);
6     for (int &v : a) scanf("%d", &v);
7     vector<int> cur = a;
8     vector<array<int, 3>> upd;
9     vector<array<int, 3>> qry;
10    for (int i = 0; i < m; i++)
11    {
12        char op;
13        int x, y;
14        scanf(" %c%d%d", &op, &x, &y);
15        x--;
16        if (op == 'Q')
17            qry.push_back({x, y, int(upd.size())});
18        else
19        {
20            upd.push_back({x, cur[x], y});
21            cur[x] = y;
22        }
23    }
24    ModifiedMo mo(n, upd.size());
25    for (auto [l, r, t] : qry) mo.add(l, r, t);
26    vector<int> cnt(1000001), res(qry.size());
27    int tot = 0;
28    auto add = [&](int p)
29    {
30        if (cnt[a[p]]++ == 0) tot++;
31    };
32    auto del = [&](int p)
33    {
34        if (--cnt[a[p]] == 0) tot--;
35    };
36    auto chg = [&](int t, bool fwd, int l, int r)
37    {
38        auto [p, pre, nxt] = upd[t];
39        bool in = l <= p && p < r;
40        if (in) del(p);
41        a[p] = fwd ? nxt : pre;
42        if (in) add(p);
43    };
44    auto ans = [&](int id)
45    {
46        res[id] = tot;
47    };
48    mo.run(add, del, chg, ans);
49    for (int v : res) printf("%d\n", v);
50 }

```

2.2.2 莫队二次离线

2.2.2.1 莫队二次离线：异或位数配对

`xor_hamming_pairs(a, queries, k, bits=14)` 按查询输入顺序返回 long long 答案, 统计 $1 \leq i < j < r$ 且 $\text{popcount}(a[i] \oplus a[j]) = k$ 的下标对。queries 为 0-based 半开区间, 允许空区间、重复查询与空数组; $0 \leq \text{bits} \leq 20$, $0 \leq a[i] < 2^{\text{bits}}$, n , m 和容器下标须可由 int 表示。k 超出 $[0, \text{bits}]$ 直接返回全零; 相等值来自不同下标才构成一对。莫队移动生成前缀事件, 第二次离线按前缀扫描; $k=0$ 显式扣掉自身。令 $C = \text{binom}(\text{bits}, k)$, 时间 $O(2^{\text{bits}} + nC + m \log(m+1) + n\sqrt{m+m})$, 空间 $O(2^{\text{bits}} + n + m)$, $m=0$ 直接返回。无需维护可逆的在线窗口, 也不支持修改。

```

1 // Queries are 0-based half-open; count unordered pairs of distinct indices.
2 vector<long long> xor_hamming_pairs(const vector<int> &a,
3                                     const vector<pair<int, int>> &queries,
4                                     int k,
5                                     int bits = 14)
6 {
7     int n = a.size(), m = queries.size();
8     vector<long long> answer(m);
9     assert(0 <= bits && bits <= 20);
10    int universe = 1 << bits;
11    for (int x : a) assert(0 <= x && x < universe);
12    for (auto [l, r] : queries) assert(0 <= l && l <= r && r <= n);
13    if (!m || k < 0 || k > bits) return answer;
14    vector<int> masks;
15    for (int x = 0; x < universe; x++)
16        if (popcount(unsigned(x)) == k) masks.push_back(x);
17    vector<int> count(universe);
18    vector<long long> prefix(n + 1);
19    for (int i = 0; i < n; i++)
20    {
21        prefix[i + 1] = prefix[i] + count[a[i]];
22        for (int mask : masks) count[a[i] ^ mask]++;
23    }
24    int block = max(1, int(n / sqrt(m)));
25    vector<int> order(m);
26    iota(order.begin(), order.end(), 0);
27    sort(order.begin(),
28         order.end(),
29         [&](int x, int y)
30         {
31             int bx = queries[x].first / block, by = queries[y].first / block;
32             if (bx != by) return bx < by;
33             return bx & 1 ? queries[x].second > queries[y].second
34                          : queries[x].second < queries[y].second;
35         });
36
37    struct Event
38    {
39        int id, l, r, sign;
40    };
41
42    vector<vector<Event>> events(n + 1);

```

莫队端点移动与前缀事件（接上页同一函数）：

```
43     int l = 0, r = 0;
44     for (int id : order)
45     {
46         auto [ql, qr] = queries[id];
47         if (r < qr)
48         {
49             events[l].push_back({id, r, qr, -1});
50             answer[id] += prefix[qr] - prefix[r];
51             r = qr;
52         }
53         if (l > ql)
54         {
55             events[r].push_back({id, ql, l, 1});
56             answer[id] -= prefix[l] - prefix[ql];
57             l = ql;
58         }
59         if (r > qr)
60         {
61             events[l].push_back({id, qr, r, 1});
62             answer[id] -= prefix[r] - prefix[qr];
63             r = qr;
64         }
65         if (l < ql)
66         {
67             events[r].push_back({id, l, ql, -1});
68             answer[id] += prefix[ql] - prefix[l];
69             l = ql;
70         }
71     }
```

第二次离线与差分还原（接上页同一函数）：

```

72     fill(count.begin(), count.end(), 0);
73     for (int p = 0; p <= n; p++)
74     {
75         if (p)
76             for (int mask : masks) count[a[p - 1] ^ mask]++;
77         for (auto e : events[p])
78             for (int i = e.l; i < e.r; i++)
79             {
80                 int value = count[a[i]] - (k == 0 && i < p);
81                 answer[e.id] += 1LL * e.sign * value;
82             }
83     }
84     long long total = 0;
85     for (int id : order)
86     {
87         total += answer[id];
88         answer[id] = total;
89     }
90     return answer;
91 }

```

模板题使用：Luogu P4887

题意：统计每个区间内异或值恰有 k 个 1 的无序下标对。输入 1-based 闭区间 $[l, r]$ ，只将 1 减一转为 0-based 半开区间； k 可以大于 14，此时全零。 $k=0$ 统计相等值的不同下标，不计自身。

所需模板：`xor_hamming_pairs`（第 20 页，2.2.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P4887>

```

1  int main()
2  {
3      ios::sync_with_stdio(false);
4      cin.tie(nullptr);
5      int n, m, k;
6      cin >> n >> m >> k;
7      vector<int> a(n);
8      for (int &x : a) cin >> x;
9      vector<pair<int, int>> queries(m);
10     for (auto &[l, r] : queries)
11     {
12         cin >> l >> r;
13         l--;
14     }
15     for (long long x : xor_hamming_pairs(a, queries, k)) cout << x << '\n';
16 }

```

第三章 配套知识与验证范围

3.1 带修莫队：时间轴与区间状态

先离线读完操作，记录每个单点修改的下标、修改前值和修改后值。询问的时间 t 表示前 t 次修改已执行；输入中的询问本身不推进时间。同一位置的第二次修改，其旧值必须来自第一次修改后的数组，不能始终使用初始值。需要离散化时，应将全部初值与修改后值一同纳入。

调度器保持三个指针 (l, r, t) ，回调统计恰好对应当前版本的半开区间 $[l, r)$ 。前进修改 i 时写入 after，回退时写入 before。若修改点在当前区间内，必须先删除旧贡献、改值、再加入新贡献；区间外只改数组。随后移动区间端点，最后按原询问编号写答案。扩展区间先于收缩，支持空区间和互不相交区间。整个 run 结束后状态停留在排序后的最后一条询问；再次调用前须重置原数组和统计量。

左端点分块、右端点分块，再按时间排序，相邻块采用蛇形顺序。时间关键字是真实修改次数，不能把询问编号当作时间维度。令块长为 B ，询问数为 Q ，修改数为 U ，则常数时间回调下的移动量上界为

$$O\left(QB + \frac{n^2}{B} + U \min(Q, (\lfloor n/B \rfloor + 1)^2)\right).$$

另有 $O(Q \log Q)$ 排序与 $O(Q)$ 调度空间。三者同阶时取 $B \approx n^{2/3}$ ，得到 $O(n^{5/3})$ ；规模悬殊时可手动调整块长。回调需要区分左右端增删时应拆分四个方向，当前简版适合不同数、频次矩等统计。

P1903 数颜色 驱动以颜色频次数组维护不同数，真实操作顺序建立修改前后值，随后交给调度器。本地快照测试逐项检查每个时间、每个区间及修改回退状态，另以顺序 set 查询核对完整驱动。最大规模验证 133333 个元素和操作，并覆盖仅修改、仅询问与重复修改同一位置。最终简短命名版本在 **P1903 记录 297653743** 通过全部 13 个测试，最慢单点 546 ms、峰值内存 14.65 MB。此记录使用默认块长与不同颜色数统计；自定义块长、空区间及任意登记时间由本地测试覆盖。最快提交榜的排名及总数尚未核验。

3.2 莫队二次离线：前缀事件与自身扣除

对区间内满足 $H(i, j) = [\text{popcount}(a_i \text{ xor } a_j) = k]$ 的无序下标对，使用 `xor_hamming_pairs`。令 $t(i) = \sum_{j < i} H(i, j)$, $P(x) = \sum_{i < x} t(i)$, 以及 $T(p, i) = \sum_{j < p} H(i, j) - [k = 0 \wedge i < p]$ 。右端扩展时贡献为 $t(i) - T(L, i)$, 左端扩展为 $T(R, i) - t(i)$ ；删除取相反数。整段端点移动的 t 和由 P 的区间差求出，剩余部分存成按前缀长度分桶的事件。第二遍顺扫只做插入，每次枚举异或后有 k 个置位的掩码更新计数；事件中的每个下标只读一次计数，并在 $k = 0$ 时扣自身。最后按莫队顺序累加窗口答案差，再按原编号返回。

区间统一为 0-based 半开区间，先扩右、扩左，再缩右、缩左，保持窗口合法。允许空区间；没有查询时直接返回。位宽默认为 14，若 k 超出位宽则答案全零。P4887 的 $k < 2^{14}$ 并不代表 $k \leq 14$ 。令 $C = \binom{b}{k}$ ，块长约为 $n/\sqrt{m}$ ，总时间 $O(2^b + nC + m \log(m+1) + n\sqrt{m} + m)$ ，空间 $O(2^b + n + m)$ 。端点移动不再乘枚举掩码数 C 。答案与前缀和使用 long long；十万个相等数的配对数超过 int32。本地全枚举、随机逐对参考及十万规模的 $k = 0, 7$ 分支通过；线上验证待补。

第一部分

跨类依赖

附录 A 数论

A.1 模算术简介

A.1.1 静态模整数

固定 int 模数 $1 \leq \text{mod} \leq \text{INT_MAX}$, 可为合数。构造接受 signed64 并正规化, v 保持 $0 \leq v < \text{mod}$; 不要手动破坏正规化。加减乘 $O(1)$, pow 非负 signed64 指数、时间 $O(\log(e+1))$ 。try_inv 用欧几里得求逆: $\text{gcd}(v, \text{mod})=1$ 时返回 optional 值, 否则 nullopt; $\text{mod}=1$ 返回代表 0。inv、/、/= 要求分母可逆; 断言开启时诊断违反前提, 关闭断言不承诺拒绝非法除法。逆元 $O(\log(\text{mod}+1))$ 、额外空间 $O(1)$, 只用 long long, 不新增辅助模板依赖。+=、-=、*=、/= 返回自身引用, 支持自赋值与链式运算, 可作为 Fenwick 区间和值类型。类型的扩展不放宽 Binomial、NTT、高斯或 FPS 自身的素数等前提。

```
1 template <int mod> struct ModInt
2 {
3     static_assert(mod >= 1);
4     int v;
5
6     ModInt(long long x = 0) : v((x % mod + mod) % mod) {}
7
8     ModInt operator+(ModInt b) const { return ModInt((long long)v + b.v); }
9
10    ModInt operator-(ModInt b) const { return ModInt((long long)v - b.v); }
11
12    ModInt operator*(ModInt b) const { return ModInt(1LL * v * b.v); }
13
14    ModInt pow(long long e) const
15    {
16        assert(e >= 0);
17        ModInt a = *this, r = 1;
18        for (; e >= 1, a = a * a)
19            if (e & 1) r = r * a;
20        return r;
21    }
```

求逆与除法（接上页同一结构体）：

```
23 optional<ModInt> try_inv() const
24 {
25     long long a = v, b = mod, x = 1, y = 0;
26     while (b)
27     {
28         long long q = a / b;
29         a -= q * b;
30         swap(a, b);
31         x -= q * y;
32         swap(x, y);
33     }
34     if (a != 1) return nullopt;
35     return ModInt(x);
36 }
37
38 ModInt inv() const
39 {
40     auto r = try_inv();
41     assert(r);
42     return *r;
43 } // requires an invertible residue
44
45 ModInt operator/(ModInt b) const { return *this * b.inv(); }
46
47 ModInt &operator+=(ModInt b) { return *this = *this + b; }
48
49 ModInt &operator-=(ModInt b) { return *this = *this - b; }
50
51 ModInt &operator*=(ModInt b) { return *this = *this * b; }
52
53 ModInt &operator/=(ModInt b) { return *this = *this / b; }
54 };
```

联合使用示例见第 31 页（Luogu P1313 计算系数），其中直接调用本模板。

A.2 卢卡斯定理

A.2.1 Lucas：小素数模的大下标组合数

Lucas(p) 预处理素数模数 p, choose(n,k) 返回 $C(n,k)$ 模 p; n、k 为完整 unsigned long long 范围, $k > n$ 返回 0, $C(0,0)=1$ 。重新赋值 Lucas(p) 可更换模数并重新预处理; 不检测素性, 要求调用者保证 p 为素数且 $p \geq 2$ 。预处理时间 $O(p + \log p)$, 动态版空间 $O(p)$; 单次查询 $O(1 + \log_p(n+1))$ 。适合小素数模数, 通常 $p \leq 10^6$; 大素数不能因查询下标小就忽略这里整段 $O(p)$ 的预处理。fac、ifac 下标为 $0..p-1$, 模数和表只读; 中间乘法用 long long。合数模数请使用 exLucas。

```

1 struct Lucas
2 {
3     int mod;
4     vector<int> fac, ifac;
5
6     Lucas(int p)
7     {
8         assert(p >= 2);
9         mod = p;
10        fac.resize(p);
11        ifac.resize(p);
12        fac[0] = 1;
13        for (int i = 1; i < p; i++) fac[i] = 1LL * fac[i - 1] * i % p;
14        long long a = fac[p - 1], r = 1;
15        for (int e = p - 2; e; e >>= 1)
16        {
17            if (e & 1) r = r * a % p;
18            a = a * a % p;
19        }
20        ifac[p - 1] = r;
21        for (int i = p - 1; i; i--) ifac[i - 1] = 1LL * ifac[i] * i % p;
22    }
23
24    int choose(unsigned long long n, unsigned long long k) const
25    {
26        if (k > n) return 0;
27        long long answer = 1;
28        while (n || k)
29        {
30            int a = n % mod, b = k % mod;
31            if (b > a) return 0;
32            answer = answer * fac[a] % mod;
33            answer = answer * ifac[b] % mod;
34            answer = answer * ifac[a - b] % mod;
35            n /= mod;
36            k /= mod;
37        }
38        return answer;
39    }
40 };

```

模板题使用：Luogu P3807

题意：多组查询 $C(n+m, n)$ 模 p ，题目保证 p 为素数且不超过 10^5 。每组构造 $\text{Lucas}(p)$ ，choose 的参数为 $n+m$ 、 n ；此题范围内 $n+m$ 不会溢出。

所需模板：Lucas（第 28 页，A.2.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3807>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int t;
6     cin >> t;
7     while (t--)
8     {
9         unsigned long long n, m;
10        int p;
11        cin >> n >> m >> p;
12        Lucas solver(p);
13        cout << solver.choose(n + m, n) << '\n';
14    }
15    return 0;
16 }
```

附录 B 组合数学

B.1 排列组合

B.1.1 阶乘组合数

代码依赖：ModInt (第 26 页, A.1.1 节)。抄写时一并准备；标准库类型不列入算法依赖。

Binomial<prime>(n=0) 预处理 $0..n$ ，要求 prime 为素数且 $0 \leq n < \text{prime}$ 。init(n) 只扩大当前上限，较小或相同的 n 不改变表；choose(n,k) 返回 $C(n,k)$ ，permute(n,k) 返回 $n!/(n-k)!$ 。查询前显式预处理到 n，查询本身不扩表。n<0, k<0 或 k>n 返回 0, $C(0,0)=P(0,0)=1$ 。fac、ifac 为可直接读取的阶乘与逆阶乘表，增长可能使旧引用失效。单次查询 $O(1)$ ；从 old 扩到 n 新增 $O(n-\text{old}+\log \text{prime})$ 次域运算，重分配还可能复制旧表。首次预处理 $O(n+\log \text{prime})$ ，空间 $O(n)$ 。超过模数不能使用阶乘除法：小素数用 Lucas，小合数用 exLucas。

```
1 template <int mod> struct Binomial
2 {
3     using Z = ModInt<mod>;
4     vector<Z> fac{Z(1)}, ifac{Z(1)};
5
6     Binomial(int n = 0) { init(n); }
7
8     void init(int n)
9     {
10         assert(0 <= n && n < mod);
11         int old = (int)fac.size() - 1;
12         if (n <= old) return;
13         fac.resize(n + 1);
14         ifac.resize(n + 1);
15         for (int i = old + 1; i <= n; i++) fac[i] = fac[i - 1] * i;
16         ifac[n] = fac[n].inv();
17         for (int i = n; i > old; i--) ifac[i - 1] = ifac[i] * i;
18     }
```

```
20 Z choose(int n, int k) const
21 {
22     if (k < 0 || k > n) return 0;
23     assert(n < (int)fac.size());
24     return fac[n] * ifac[k] * ifac[n - k];
25 }
26
27 Z permute(int n, int k) const
28 {
29     if (k < 0 || k > n) return 0;
30     assert(n < (int)fac.size());
31     return fac[n] * ifac[n - k];
32 }
33 };
```

应用题补充：Luogu P1313 计算系数

题意：求 $(by+ax)^k$ 的 k 次方中指定项的系数，模 10007。 x 、 y 的指数分别为 n 、 m ，均非负且 $n+m=k$ 。 k 至多 1000， a 、 b 均为不超过 1000000 的非负数。组合 $C(k,n)$ 与两个幂即可。模数为素数且 k 小于模数，满足阶乘逆元前提；零次幂按空乘积为 1。

所需模板：ModInt（第 26 页，A.1.1 节）、Binomial（第 30 页，B.1.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 std 命名空间。

题目：<https://www.luogu.com.cn/problem/P1313>

```
1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int a, b, k, n, m;
6     cin >> a >> b >> k >> n >> m;
7     Binomial<10007> c(k);
8     auto x = ModInt<10007>(a).pow(n);
9     auto y = ModInt<10007>(b).pow(m);
10    cout << (c.choose(k, n) * x * y).v << '\n';
11 }
```

附录 C 树状数组

C.1 树状数组第 k 小与组合依赖

保留频数第 k 小和泛型组合用途；P3374 基础单点加/区间和题不再独立收录或安排提交。add(x,v)、sum(x)、query(l,r)，下标 1..n，闭区间；kth(k) 要求所有频率非负且 k>0，超过总和返回 n+1。单次 O(log n)。T 须能从 0 构造并支持 += 与减法；可直接使用 ModInt 维护模数下的区间和。kth 只适用于有序累计频数，不适用于取模后的区间和。

```
1 template <class T = long long> struct Fenwick
2 {
3     int n;
4     vector<T> a;
5
6     Fenwick(int n = 0) : n(n), a(n + 1) {}
7
8     void add(int x, T v)
9     {
10         assert(x >= 1);
11         for (; x <= n; x += x & -x) a[x] += v;
12     }
13
14     T sum(int x) const
15     {
16         T ans = 0;
17         for (; x; x -= x & -x) ans += a[x];
18         return ans;
19     }
20
21     T query(int l, int r) const { return sum(r) - sum(l - 1); }
22
23     // Nonnegative frequencies; returns n+1 if k exceeds total.
24     int kth(T k) const
25     {
26         assert(k > 0);
27         int x = 0, b = 1;
28         while (b <= n / 2) b *= 2;
29         for (; b >= 1)
30             if (x + b <= n && a[x + b] < k)
31             {
32                 k -= a[x + b];
33                 x += b;
34             }
35         return x + 1;
36     }
37 };
```

模板题使用：Luogu P3369

题意：维护可重集合的插入、删除一个、排名、第 k 小和严格前驱/后继。预读全部操作并离散化非第 k 小查询的值域；频数下标为 1-based, k th 的返回值减 1 后映射回原值。排名查询允许 x 不在集合中；第 k 小与前驱/后继按题目保证存在。 $n \leq 100000$, $|x| \leq 10000000$ ；总复杂度 $O(n \log n)$ ，空间 $O(n)$ 。

所需模板：Fenwick（第 32 页，C.1 节）。以下仅含使用代码，默认已粘贴所需模板并包含标准头文件、使用 `std` 命名空间。

题目：<https://www.luogu.com.cn/problem/P3369>

```

1 int main()
2 {
3     ios::sync_with_stdio(false);
4     cin.tie(nullptr);
5     int n;
6     if (!(cin >> n)) return 0;
7     vector<pair<int, int>> q(n);
8     vector<int> a;
9     for (auto &[op, x] : q)
10    {
11        cin >> op >> x;
12        if (op != 4) a.push_back(x);
13    }
14    sort(a.begin(), a.end());
15    a.erase(unique(a.begin(), a.end()), a.end());
16    Fenwick<int> f(a.size());
17    for (auto [op, x] : q)
18    {
19        int p = lower_bound(a.begin(), a.end(), x) - a.begin();
20        if (op == 1)
21            f.add(p + 1, 1);
22        else if (op == 2)
23        {
24            if (f.query(p + 1, p + 1) > 0) f.add(p + 1, -1);
25        }
26        else if (op == 3)
27            cout << f.sum(p) + 1 << '\n';
28        else if (op == 4)
29            cout << a[f.kth(x) - 1] << '\n';
30        else if (op == 5)
31            cout << a[f.kth(f.sum(p)) - 1] << '\n';
32        else
33        {
34            p = upper_bound(a.begin(), a.end(), x) - a.begin();
35            cout << a[f.kth(f.sum(p) + 1) - 1] << '\n';
36        }
37    }
38 }

```

算法索引

Binomial, 30

Chain3D, 12

Dominance3D, 10

ExactCover, 1

Fenwick, 32

Lucas, 28

MinimumCover, 5

ModifiedMo, 17

ModInt, 26

xor_hamming_pairs, 20

带修莫队, 23